Enterprise Data Ingestion Architecture for Nigerian Economic and Energy Intelligence RAG Pipelines

The foundation of a highly effective Retrieval-Augmented Generation (RAG) system lies not in the complexity of the large language model, but in the fidelity, recency, and breadth of the data it retrieves. For a system tasked with providing deep intelligence on the Nigerian macroeconomic landscape, the Gulf of Guinea maritime sector, and the broader global energy market, static datasets are entirely insufficient. The pipeline must be continuously hydrated with top-grade, authoritative, and multi-dimensional information.

This document outlines a comprehensive, automated data engineering framework designed to ingest open-source, zero-cost intelligence across five foundational data pillars. Furthermore, it details the advanced network evasion techniques required to extract data from aggressively protected governmental and financial portals, providing the architectural blueprints and executable ingestion scripts necessary to operationalize this pipeline from external geographic locations, specifically addressing the technical hurdles of operating from servers located in Mississauga, Ontario.

1. The Network Evasion Layer: Ontario to Nigeria Data Corridors

Before detailing the specific intelligence pillars, the structural reality of cross-continental data ingestion must be addressed. Attempting to ingest data from Nigerian government domains (.gov.ng), financial exchange portals, and global market aggregators using standard Python HTTP libraries (such as urllib or the standard requests library) from a datacenter in Mississauga, Ontario, will result in immediate and persistent failure. These domains deploy sophisticated anti-bot countermeasures, predominantly managed by Cloudflare Web Application Firewalls (WAF) and similar traffic mitigation services.[1]

1.1. The Mechanics of Modern Bot Detection

Cloudflare and other perimeter defense systems no longer rely on simple User-Agent string validation. They employ deep packet inspection and browser fingerprinting to differentiate automated scripts from legitimate human users navigating the web.[1] Key detection vectors include:

- **TLS Fingerprinting (JA3/JA4):** The initial cryptographic handshake between a client and a server contains a specific configuration of cipher suites, extensions, and elliptic curves. Standard Python HTTP libraries generate a cryptographic signature that explicitly identifies them as automated scripts rather than modern web browsers like Google Chrome or Mozilla Firefox.[1]

- **HTTP/2 Frame Multiplexing:** Modern browsers handle HTTP/2 connections in highly specific ways, multiplexing requests across single connections with specific header orderings (using pseudo-headers like :method and :authority). Standard scraping libraries often fail to replicate these exact frame sequences, triggering an immediate anomaly flag.
- **JavaScript Execution and Browser Variables:** Cloudflare's Turnstile and interstitial challenges (the "Verify you are human" pages) require the client browser to solve complex background JavaScript puzzles.[2] Furthermore, headless browsers using standard Selenium expose internal variables such as navigator.webdriver = true, immediately flagging the connection as an automated testing framework.[3]
- **Geographic and Autonomous System Number (ASN) Filtering:** Connections originating from North American cloud hosting providers (e.g., AWS, Azure, GCP datacenters in Canada) accessing localized Nigerian municipal or regulatory data at high frequencies are routinely classified as anomalous traffic and subjected to geographic rate limiting.

1.2. Engineering the Circumvention Strategy

To successfully run an automated ingestion pipeline from Mississauga, the extraction architecture must be heavily fortified using a multi-tiered evasion approach.
For extracting REST API payloads, downloadable PDFs, and static HTML content, invoking a full headless browser is computationally wasteful and heavily strains server memory. Instead, the modern approach utilizes curl_cffi.[2] This library acts as a Python binding for curl but relies specifically on Google Chrome's BoringSSL library rather than standard OpenSSL. This perfectly mimics the TLS fingerprints of a real browser, allowing the script to bypass standard Cloudflare WAF rules without the overhead of rendering the Document Object Model (DOM).[2]
For portals that force a Cloudflare Turnstile challenge or dynamically load content via heavily obfuscated JavaScript—such as the Nigerian Exchange Group (NGX) portal or peer-to-peer cryptocurrency platforms—a browser environment is mandatory. The pipeline must leverage modern alternatives to standard Selenium:

- **SeleniumBase (UC Mode):** An advanced toolkit that implements "Undetected Chromedriver" logic. It automatically scrubs the navigator.webdriver flags, modifies the browser environment to appear human, and handles CAPTCHA clicks natively.[2]
- **Nodriver and Camoufox:** Nodriver is a highly performant, asynchronous Python browser automation library built on the Chrome DevTools Protocol (CDP) that natively avoids standard bot markers.[1] Camoufox offers a heavily patched Firefox build specifically designed to spoof hardware concurrency, WebGL fingerprints, and audio contexts to defeat advanced canvas fingerprinting.[4]

Finally, the pipeline must integrate high-quality, rotating residential proxies.[1] By routing requests originating from Ontario through IP addresses geographically localized to West Africa or neutral European residential hubs, the WAF trust scores remain sufficiently high to avoid continuous Turnstile interrogations.

2. Pillar I: Sovereign Macroeconomics, Monetary Policy, and Debt Dynamics

To contextualize any corporate or energy data, the RAG pipeline must continuously ingest macroeconomic indicators that dictate domestic purchasing power, fiscal policy, and capital allocation in Nigeria.

2.1. National Bureau of Statistics (NBS)

The NBS is the definitive repository for domestic economic indicators. In 2025, the NBS finalized the rebasing of the Consumer Price Index (CPI) and Gross Domestic Product (GDP), updating the price reference period to 2024 to accurately reflect current economic realities.[5]
The NBS publishes detailed monthly CPI and Inflation reports. For instance, as of January 2026, the headline inflation rate eased slightly to 15.10%, down from 15.15% in December 2025, marking a continuous ten-month decline.[7] Food inflation, the largest component of the basket, fell to 8.89%, while core inflation moderated to 17.72%.[7] The NBS microdata portal hosts these monthly reports as downloadable PDFs, alongside Microsoft Excel tables and ZIP archives containing granular price watches for specific commodities like Automotive Gas Oil (Diesel) and Premium Motor Spirit (Petrol).[5]
Ingesting these PDFs and translating them into vectorizable Markdown allows the RAG model to answer complex questions regarding the real yield of Nigerian corporate debt or the impact of diesel prices on supply chain logistics.

2.2. Central Bank of Nigeria (CBN)

Monetary policy dictates capital liquidity and foreign exchange stability. The CBN publishes Monetary Policy Communiques detailing the decisions of the Monetary Policy Committee (MPC). Tracking these shifting parameters is essential for any financial intelligence tool.

MPC Meeting Date	MPR	CRR (Commercial)	Asymmetric Corridor	Key Context / Decisions
Feb 23-24, 2026	26.50%	45.00%	+50/-450 bps	Rate cut by 50 bps. Signifies confidence in easing inflation.[9]
Nov 24-25, 2025	27.00%	45.00%	+50/-450 bps	Rates held steady; liquidity ratio maintained at 30.00%.[9]
Sep 22-23, 2025	27.00%	45.00%	+250/-250 bps	Introduction of a 75% CRR on non-TSA public sector deposits.[9]

July 21-22, 2025	27.50%	50.00%	+500/-100 bps	Retained tight monetary stance to combat mid-year inflation peaks.[9]

Beyond communiques, the CBN maintains an extensive statistical bulletin covering external reserves, M3 money supply, and exchange rate metrics.[10] The RAG pipeline must scrape the CBN's document directories to capture these historical shifts.

2.3. Debt Management Office (DMO)

National debt obligations severely impact a sovereign's ability to fund infrastructure. The Debt Management Office (DMO) of Nigeria publishes comprehensive public debt statistical bulletins and quarterly statements.[11] The DMO portal provides PDFs detailing the total public debt, external debt stock, and debt service metrics, alongside daily updates on Nigeria's Eurobonds closing prices and yields.[11] For instance, the DMO releases aggregated debt profiles, such as the total public debt as of December 31, 2024, which allows the RAG system to calculate debt-to-GDP ratios when cross-referenced with NBS data.[12]

3. Pillar II: Upstream Hydrocarbon Production & Regulatory Audits

The Nigerian economy is intrinsically tied to upstream petroleum production. Acquiring primary documentation from national regulators is mandatory for accurate energy market modeling and forecasting.

3.1. Nigerian Upstream Petroleum Regulatory Commission (NUPRC)

The NUPRC is the definitive source for upstream operational metrics, acreage management, and regulatory frameworks. The commission actively gazettes critical regulations that alter the operational landscape and cost structure for upstream operators.

Key Gazetted Regulations for Ingestion:

Regulation Title	Last Updated	Implications for Intelligence
Nigeria Upstream Petroleum Commercial Regulations	2025	Dictates commercial frameworks, field development plans, and joint venture cost structures.[14]
Upstream Petroleum Fees and Rents Temporary Regulations	2025	Critical for calculating operational overhead for E&P companies.[14]
Nigerian Upstream Petroleum Safety Regulations	2024	Mandates compliance standards, impacting facility

		design and maintenance capital expenditures.[14]
Significant Crude Oil and Gas Discovery Regulations	2023	Provides the legal framework for reporting new reserves, impacting national asset valuations.[14]
Domestic Gas Delivery Regulations	2022	Outlines obligations for upstream operators to supply the domestic market.[14]

Furthermore, the NUPRC releases vital production forecasts. The *1st Half 2025 Domestic Crude Oil Supply Obligation (DCSO) Refinery Requirement & Production Forecast* dictates that functional refineries in Nigeria require 770,500 barrels per day (bopd) for the first half of 2025.[14] The NUPRC forecasts an average daily production of 2,066,940 bopd for this period, breaking down expected contributions from major operators such as Shell Petroleum (SPDC) at 226,421 bpd, MPNU/Seplat at 216,462 bpd, and Chevron Nigeria at 190,169 bpd.[14] Integrating this specific data allows the RAG model to audit whether actual production meets the statutory domestic supply obligations.

3.2. Nigerian National Petroleum Company Limited (NNPC)

As the state-owned oil corporation, NNPC Limited operates as a primary commercial entity. The pipeline must pull its Audited Financial Statements (AFS) and investor presentations from the NNPC investor relations filesystem.[16] For the fiscal year 2024, NNPC reported massive growth, declaring a Profit After Tax (PAT) of ₦5.4 trillion on revenue of ₦45.1 trillion.[16] The company also proposed total dividends of ₦4.3 trillion, representing a 64% increase over 2023.[16] By extracting the raw text from these PDFs, the RAG model gains the ability to analyze NNPC's operational efficiency, cost discipline, and its strategic capital investments in gas infrastructure (such as the AKK pipeline).[20]

3.3. Nigeria Extractive Industries Transparency Initiative (NEITI)

To ensure the intelligence is not solely reliant on self-reported data, the pipeline must ingest independent audits. NEITI publishes comprehensive reports assessing and reconciling the physical, process, and financial flows within the oil and gas sector.[21] The final reports for the 2022 and 2023 industry audits, released in late 2024, cover critical data on revenue collection, unremitted funds, and unauthorized deductions (such as historical kerosene subsidies).[21] These documents provide the RAG system with the necessary context to identify fiscal leakages and evaluate sovereign transparency.

4. Pillar III: Global Energy Pricing, Downstream Operations, and Corporate Debt

Nigerian crude valuation and refining economics cannot be analyzed in a vacuum. They are

dependent on global pricing benchmarks and domestic downstream regulatory frameworks.

4.4. Nigerian Midstream and Downstream Petroleum Regulatory Authority (NMDPRA)

For downstream logistics, pipeline tariffs, and refining parameters, the NMDPRA portal hosts essential guidelines. Key documents for ingestion include the *Midstream and Downstream Petroleum Operations Regulations 2022*, the *Gas Pricing Domestic Demand and Delivery Regulations*, and specific manuals for petroleum product importation permits.[25] These regulations govern the operational realities of massive infrastructure projects like the Dangote Refinery.

4.5. Corporate Debt and NGX Filings

Corporate filings provide micro-level validation of the macroeconomic trends reported by the NBS and CBN. The Nigerian Exchange Group (NGX) Issuers Portal hosts regulatory filings and corporate actions.[28] A prime example to target is the debt instruments issued by major industrial players. In early 2026, Dangote Cement listed Series 1 and Series 2 Commercial Papers (CPs) with a combined value of ₦119.87 billion.[29] The 181-day Series 1 CP offered an implied yield of 17.50%, while the 265-day Series 2 offered 19.00%.[29]

By scraping these yields, the RAG system can dynamically compare corporate borrowing costs against the CBN's MPR of 26.5% [9] and the inflation rate of 15.10% [7], allowing for highly sophisticated, automated yield curve analysis. The NGX Market Data API provides a free tier granting 7-day historical price data, which can be queried to update corporate equity valuations.[30]

4.6. Global Energy Benchmarks (OPEC and EIA)

Global supply and demand metrics dictate the baseline value of Nigerian exports. The Organization of the Petroleum Exporting Countries (OPEC) publishes its Monthly Oil Market Report (MOMR), an exhaustive PDF detailing global economics, oil demand, and OPEC+ production quotas.[31] The January and February 2026 reports, for example, maintained a global oil demand growth forecast of 1.4 million barrels per day (mb/d) for the year, while noting a plunge in world oil supply by 1.2 mb/d in January due to severe weather and outages.[32] Extracting the data from the MOMR allows the RAG system to contextualize Nigeria's production quotas against global shortfalls.

Simultaneously, the profitability of Nigerian refineries depends on the spread between raw crude costs and refined product prices. The U.S. Energy Information Administration (EIA) provides free, robust datasets on spot prices for Brent crude, WTI, RBOB gasoline, and ultra-low sulfur diesel (ULSD).[34]

To calculate these real-time refining margins, the ingestion pipeline can connect to free commodity pricing APIs such as **OilPriceAPI** or **Commodities-API**.[36] OilPriceAPI offers REST API endpoints for real-time crude oil prices, refined products, and crack spreads with ultra-low latency.[36] The fundamental formula for a 3:2:1 crack spread, highly relevant to forecasting

margins for the Dangote Refinery, relies on these real-time data feeds:

---

5. Pillar IV: Maritime Telemetry and Port Logistics

Physical commodity movement serves as an unforgeable leading indicator for economic output. The Gulf of Guinea is a critical maritime corridor, and tracking vessel movements provides real-time, empirical validation of the crude export metrics published by the NUPRC and the refined product import data tracked by the NMDPRA.

5.1. Automatic Identification System (AIS) Telemetry

While premium live vessel tracking data is heavily commercialized, robust open-source and free-tier solutions exist. The platform **aisstream.io** provides a free WebSocket API that streams terrestrial AIS data globally.[39] By establishing a geographic bounding box specifically around the Nigerian coastline and the Gulf of Guinea (e.g., Latitudes -2.0 to 7.0, Longitudes 2.0 to 10.0), the pipeline can capture high-frequency PositionReport and ShipStaticData messages.[39] This telemetry includes Maritime Mobile Service Identity (MMSI) numbers, speed, course, and navigational status, allowing the RAG system to physically track oil tankers leaving the Niger Delta terminals.

An alternative source is **AISHub**, which grants full API access to aggregated terrestrial feeds for users who contribute their own local AIS antenna data.[41] Utilizing these feeds enables the system to monitor supply chain velocity without exorbitant data licensing fees.

5.2. Lekki Deep Sea Port Schedules

The Lekki Freeport Terminal, located in the Lagos Free Zone, represents the vanguard of Nigerian maritime logistics and is geographically proximate to the Dangote Refinery. The terminal's online vessel schedule interface lists the Estimated Time of Arrival (ETA) and Estimated Time of Departure (ETD) for major cargo and bunkering vessels.[43]

Recent tracking data shows arrivals of tugs and pilot vessels like the *ALYASAT-1*, *MT.IRAGBIJI*, and the bulk carrier *ADASTRA*.[44] Tracking platforms like **Terminal49** or **VesselFinder** offer free tiers or basic HTML interfaces that can be scraped to track container availability and dwell times at the Lekki port.[45] Comparing the scheduled ETA against the actual arrival time provides the LLM with empirical data regarding port congestion and supply chain bottlenecks.

6. Pillar V: Geopolitics, Conflict, and Alternative Financials

The Nigerian energy and economic landscape is profoundly sensitive to geopolitical instability, regional conflicts, and currency volatility.

6.1. Armed Conflict Location & Event Data Project (ACLED)

ACLED represents the gold standard for independent, near-real-time political violence data.[47] It meticulously tracks battles, violence against civilians, and demonstrations.[48] This data is freely accessible via the Humanitarian Data Exchange (HDX) platform using the hdx-python-api library, or directly through the ACLED API using OAuth or cookie-based authentication.[49] Ingesting this dataset—which is frequently updated in CSV and XLSX formats—allows the RAG system to correlate localized conflict events in the oil-producing Niger Delta or the agricultural belts in the North with subsequent economic outcomes.[47] If the data shows a spike in kinetic events involving pipeline vandalism, the model can instantly predict an impending drop in the NUPRC's monthly crude production numbers.

6.2. Parallel Market Foreign Exchange Rates

The official CBN exchange rate rarely reflects the actual street value of the Naira, a discrepancy that fundamentally impacts inflation and the cost of imported goods. Capturing peer-to-peer (P2P) cryptocurrency exchange rates (e.g., USDT/NGN on Binance) acts as a highly accurate, real-time proxy for the parallel market rate.[51] Because this data is actively defended against scrapers by Cloudflare, it must be extracted programmatically using advanced browser automation tools like SeleniumBase running in headless UC mode.[2]

---

7. Comprehensive Python Ingestion Framework

The following section provides the robust, production-ready Python architecture required to ingest this vast array of intelligence from Mississauga, Ontario. The design is highly modular, isolating the network evasion logic from the specific endpoint parsing logic to ensure maintainability.

7.1. Environment Configuration

To execute these scripts, specific libraries designed for network evasion, API consumption, and PDF parsing must be installed.

Bash

```
# Install required dependencies
pip install curl_cffi seleniumbase websockets hdx-python-api pdfplumber asyncio aiohttp pandas beautifulsoup4
```

7.2. The Network Evasion Base Client

This base class provides the fundamental transport layer for extracting static files and APIs,

utilizing Chrome TLS impersonation to bypass initial WAF blocks on Nigerian and global domains.

Python

```python
# data_engineering/ingestion/base_client.py
from curl_cffi import requests
import logging
from typing import Optional, Dict, Any
import os

class StealthClient:
    """
    A fortified HTTP client that mimics modern browser TLS fingerprints (JA3/JA4)
    to bypass WAFs and anti-bot protections on target governmental portals.
    """
    def __init__(self, proxy_url: Optional[str] = None):
        logging.basicConfig(level=logging.INFO)
        self.logger = logging.getLogger(__name__)
        # Utilize rotating residential proxies to bypass geographic rate limiting
        self.proxies = {"http": proxy_url, "https": proxy_url} if proxy_url else None

        # Impersonate a modern Chrome browser to generate a legitimate TLS fingerprint
        self.session = requests.Session(impersonate="chrome120", proxies=self.proxies)
        self.session.headers.update({
            "Accept-Language": "en-US,en;q=0.9",
            "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8",
            "Upgrade-Insecure-Requests": "1",
            "Sec-Fetch-Dest": "document",
            "Sec-Fetch-Mode": "navigate",
            "Sec-Fetch-Site": "none",
            "Sec-Fetch-User": "?1"
        })

    def fetch_get(self, url: str, params: Optional] = None) -> requests.Response:
        try:
            self.logger.info(f"Fetching: {url}")
            response = self.session.get(url, params=params, timeout=45)
            response.raise_for_status()
            return response
        except Exception as e:
            self.logger.error(f"Failed to fetch {url}. Potential WAF Block or Timeout. Error: {e}")
```

```
        raise

    def download_file(self, url: str, save_path: str):
        """Streams large binary payloads (PDFs, ZIPs) to disk."""
        self.logger.info(f"Initiating download from {url} to {save_path}")
        response = self.session.get(url, stream=True, timeout=60)
        response.raise_for_status()

        os.makedirs(os.path.dirname(save_path), exist_ok=True)
        with open(save_path, 'wb') as f:
            for chunk in response.iter_content(chunk_size=8192):
                if chunk:
                    f.write(chunk)
        self.logger.info(f"Successfully downloaded artifact to {save_path}")
```

7.3. Macroeconomic and Energy Policy Extractor

This module handles the extraction and parsing of complex PDFs from the CBN, NUPRC, and DMO, converting them into Markdown tables suitable for RAG embedding.

Python

```
# data_engineering/ingestion/macro_energy_extractor.py
import json
import os
from bs4 import BeautifulSoup
import pdfplumber
from base_client import StealthClient

class InstitutionalDataExtractor:
    def __init__(self, storage_dir: str, proxy: str = None):
        self.client = StealthClient(proxy_url=proxy)
        self.storage_dir = storage_dir
        os.makedirs(self.storage_dir, exist_ok=True)

    def extract_cbn_mpc_communiques(self):
        """
        Scrapes the Central Bank of Nigeria for the latest Monetary Policy decisions.
        """
        url = "https://www.cbn.gov.ng/MonetaryPolicy/decisions.html"
        html_content = self.client.fetch_get(url).text
        soup = BeautifulSoup(html_content, 'html.parser')
```

```
        decisions =
        # Target specific structural elements holding the communique summaries
        for paragraph in soup.find_all('p'):
            text = paragraph.get_text(strip=True)
            # Filter for specific semantic markers indicating an MPC rate decision
            if "The Committee decided at the" in text and "meeting of the MPC held" in text:
                decisions.append(text)

        output_path = os.path.join(self.storage_dir, "cbn_mpc_latest.md")
        with open(output_path, "w", encoding='utf-8') as f:
            f.write("# Central Bank of Nigeria - Recent MPC Decisions\n\n")
            for d in decisions:
                f.write(f"- {d}\n\n")
        print(f"Extracted {len(decisions)} CBN MPC records to {output_path}.")

    def extract_dmo_debt_statistics(self):
        """
        Locates and downloads the latest Public Debt Statistical Bulletins from the DMO.
        """
        url =
"https://www.dmo.gov.ng/publications/other-publications/nigeria-s-public-debt-statistical-bull
etin"
        html = self.client.fetch_get(url).text
        soup = BeautifulSoup(html, 'html.parser')

        for link in soup.find_all('a', href=True):
            if 'download' in link['href'].lower() and 'bulletin' in link.get_text(strip=True).lower():
                title = link.get_text(strip=True).replace(" ", "_").replace("/", "-")
                full_url = f"https://www.dmo.gov.ng{link['href']}"
                file_path = os.path.join(self.storage_dir, f"DMO_{title}.pdf")

                print(f"Downloading DMO Data: {title}")
                self.client.download_file(full_url, file_path)

                # Automatically parse the downloaded PDF into Markdown
                self.parse_local_pdf_to_markdown(file_path)

    def parse_local_pdf_to_markdown(self, file_path: str) -> str:
        """
        Converts downloaded PDFs (e.g., NBS CPI reports, DMO bulletins) into Markdown,
        preserving table structures for superior LLM semantic chunking.
        """
```

```
    md_text = f"# Parsed Document: {os.path.basename(file_path)}\n\n"
    try:
        with pdfplumber.open(file_path) as pdf:
            for page in pdf.pages:
                # Extract raw narrative text
                text = page.extract_text()
                if text:
                    md_text += f"{text}\n\n"

                # Extract and format complex tables as Markdown
                tables = page.extract_tables()
                for table in tables:
                    if not table: continue
                    for i, row in enumerate(table):
                        # Clean newline characters within cells
                        cleaned_row = [str(cell).replace('\n', ' ') if cell else "" for cell in row]
                        md_text += "| " + " | ".join(cleaned_row) + " |\n"
                        # Add markdown header separator after the first row
                        if i == 0:
                            md_text += "|" + "|".join(["---"] * len(row)) + "|\n"
                    md_text += "\n"

        # Save the markdown representation next to the PDF
        md_path = file_path.replace('.pdf', '.md')
        with open(md_path, 'w', encoding='utf-8') as f:
            f.write(md_text)
        print(f"Successfully parsed {file_path} into Markdown.")
        return md_text
    except Exception as e:
        print(f"Error parsing PDF {file_path}: {e}")
        return ""
```

7.4. Dynamic Content Evasion: Parallel FX and NGX Data

For complex portals heavily reliant on JavaScript rendering, Cloudflare Turnstile, or dynamic data loading (such as Binance P2P for NGN/USDT rates, or the NGX Issuers portal), curl_cffi is insufficient. The pipeline transitions to SeleniumBase running in Undetected Chromedriver (UC) mode to execute a real browser instance seamlessly.

Python

```
# data_engineering/ingestion/dynamic_market_scraper.py
```

```
from seleniumbase import SB
import json
import os
import time

class DynamicMarketExtractor:
    def __init__(self, storage_dir: str):
        self.storage_dir = storage_dir
        os.makedirs(self.storage_dir, exist_ok=True)

    def extract_parallel_fx_rates(self):
        """
        Uses SeleniumBase UC mode to bypass Cloudflare Turnstile and
        extract live NGN/USDT P2P exchange rates from Binance.
        """
        print("Initializing Undetected Chromedriver for P2P FX scraping...")
        # UC mode masks webdriver presence and solves JS challenges automatically
        with SB(uc=True, headless=True) as sb:
            url = "https://p2p.binance.com/en/trade/all-payments/USDT?fiat=NGN"

            # Open with reconnect logic to bypass initial WAF interstitial pages
            sb.uc_open_with_reconnect(url, reconnect_time=4)

            # Wait for dynamic DOM elements representing price rows to load
            try:
                # The exact CSS selector may require periodic updating based on site changes
                sb.wait_for_element_visible("div.css-1m1f8hn", timeout=20)

                # Extract raw text from price elements
                prices = sb.find_elements("div.css-1m1f8hn")
                extracted_rates = [p.text.strip() for p in prices[:5] if p.text.strip()]

                result = {
                    "asset": "USDT/NGN",
                    "timestamp": time.time(),
                    "top_5_rates": extracted_rates
                }

                out_path = os.path.join(self.storage_dir, "parallel_fx_rates.json")
                with open(out_path, "w") as f:
                    json.dump(result, f, indent=4)

                print("Successfully Extracted Latest NGN/USDT Parallel Market Rates.")
```

```
        return result

    except Exception as e:
        print(f"Failed to locate dynamic elements. WAF block or DOM change: {e}")
```

7.5. The Maritime Telemetry Pipeline (WebSocket Stream)

AIS data operates on continuous streams. To prevent memory overflow and to chunk data appropriately for a RAG vector store, the WebSocket client must aggregate spatial data dynamically using an asynchronous event loop.

Python

```
# data_engineering/ingestion/ais_telemetry_stream.py
import asyncio
import websockets
import json
import os
from datetime import datetime

class GulfOfGuineaAISStreamer:
    """
    Subscribes to aisstream.io to monitor real-time vessel traffic
    in the Gulf of Guinea and Lekki Port approaches.
    """
    def __init__(self, api_key: str, storage_dir: str):
        self.ws_url = "wss://stream.aisstream.io/v0/stream"
        self.api_key = api_key
        self.storage_dir = storage_dir
        os.makedirs(self.storage_dir, exist_ok=True)
        self.file_path = os.path.join(self.storage_dir, "maritime_telemetry.jsonl")

        # Bounding box covering the Nigerian Coastline and Gulf of Guinea
        # Format: [[Lat_Min, Lon_Min], [Lat_Max, Lon_Max]]
        self.bounding_box = [[2.0, 2.0], [7.5, 9.5]]

    async def stream_data(self):
        async with websockets.connect(self.ws_url) as websocket:
            subscription = {
                "APIKey": self.api_key,
                "BoundingBoxes": [self.bounding_box],
                "FilterMessageTypes":
```

```
        }
        await websocket.send(json.dumps(subscription))
        print(f"Connected to AIS Stream. Monitoring bounding box: {self.bounding_box}")

        event_buffer =

        while True:
            try:
                message = await asyncio.wait_for(websocket.recv(), timeout=60.0)
                data = json.loads(message)

                if data.get("MessageType") == "PositionReport":
                    msg_info = data.get("Message", {}).get("PositionReport", {})
                    mmsi = data.get("MetaData", {}).get("MMSI")
                    ship_name = data.get("MetaData", {}).get("ShipName", "UNKNOWN")

                    record = {
                        "timestamp": datetime.utcnow().isoformat(),
                        "mmsi": mmsi,
                        "ship_name": ship_name.strip(),
                        "latitude": msg_info.get("Latitude"),
                        "longitude": msg_info.get("Longitude"),
                        "speed_over_ground": msg_info.get("Cog"),
                        "navigational_status": msg_info.get("NavigationalStatus")
                    }
                    event_buffer.append(record)

                    # Flush to disk every 50 records for RAG embedding batching
                    if len(event_buffer) >= 50:
                        self._flush_to_disk(event_buffer)
                        event_buffer.clear()

            except asyncio.TimeoutError:
                print("AIS Stream Heartbeat Timeout. Attempting to reconnect...")
                break
            except Exception as e:
                print(f"WebSocket Error: {e}")
                break

def _flush_to_disk(self, buffer: list):
    # Append JSON lines to a local file for later vectorization
    with open(self.file_path, "a") as f:
        for item in buffer:
```

```
            f.write(json.dumps(item) + "\n")
        print(f"Flushed {len(buffer)} AIS position records to storage.")
```

7.6. Geopolitical Data Ingestion via HDX API

Retrieving conflict data requires programmatic parameters rather than manual scraping to ensure consistency. The pipeline utilizes the official hdx-python-api to pull ACLED data from the Humanitarian Data Exchange.

Python

```
# data_engineering/ingestion/geopolitics_extractor.py
from hdx.utilities.easy_logging import setup_logging
from hdx.hdx_configuration import Configuration
from hdx.data.dataset import Dataset
import pandas as pd
import os

class ACLEDDataExtractor:
    """
    Interacts with the HDX API to pull ACLED conflict datasets for Nigeria,
    transforming tabular data into Markdown for LLM comprehension.
    """
    def __init__(self, api_key: str, storage_dir: str):
        self.storage_dir = storage_dir
        os.makedirs(self.storage_dir, exist_ok=True)

        setup_logging()
        # Initialize HDX Configuration
        Configuration.create(hdx_site='prod', user_agent='RAG_Ingestion_Pipeline',
hdx_key=api_key)

    def download_nigeria_conflict_data(self):
        print("Querying HDX API for ACLED Nigeria Conflict Data...")
        # Targeting the specific dataset identifier for Nigeria ACLED data
        dataset = Dataset.read_from_hdx('nigeria-acled-conflict-data')

        if not dataset:
            print("Failed to locate dataset on HDX.")
            return

        resources = dataset.get_resources()
```

```
for resource in resources:
    # Target structured data files
    if 'xlsx' in resource['format'].lower() or 'csv' in resource['format'].lower():
        file_name = resource['name']
        print(f"Downloading ACLED resource: {file_name}")

        target_path = os.path.join(self.storage_dir, file_name)
        # HDX library handles the download stream automatically
        url, path = resource.download(self.storage_dir)

        # Transform to Markdown for optimal LLM context window ingestion
        if path.endswith('.xlsx'):
            df = pd.read_excel(path)
            # Convert the top 100 most recent events to markdown
            md_table = df.head(100).to_markdown(index=False)
            md_path = path.replace('.xlsx', '.md')

            with open(md_path, 'w', encoding='utf-8') as f:
                f.write(f"# ACLED Conflict Data Extract\n\n{md_table}")
            print(f"Generated Markdown table for RAG ingestion at {md_path}")
```

---

8. Strategic Synthesis and RAG Integration Architecture

The act of merely accumulating this data into a centralized storage bucket does not generate intelligence. The true power of this data engineering framework is realized in how these disparate, multi-format data pillars are processed, chunked, and structured within the vector database to facilitate cross-domain synthesis by the Large Language Model.

8.1. Semantic Chunking and Metadata Injection

Data generated by the NUPRC regarding domestic crude supply obligations (DCSO) operates on a vastly different time scale (monthly/annual projections) [14] than the real-time websocket data from maritime AIS systems.[39] If these files are blindly chunked by character count and embedded, the LLM will hallucinate relationships due to temporal misalignment.

When the parsing scripts convert the PDFs and JSON lines into text chunks for the embedding model, aggressive metadata injection is critical. Every text chunk sent to the vector database must be tagged with rigorous key-value pairs:

- {"domain": "macroeconomics", "timestamp": "2026-02-24", "source": "CBN_MPC", "valid_until": "2026-03-24"}

- {"domain": "maritime_telemetry", "timestamp": "2026-03-08T13:05:00", "source": "AIS_Lekki", "vessel_mmsi": "MT.IRAGBIJI"}

With this architecture, if a user queries the RAG system with, "Why did the Dangote Commercial Paper yield 19%?", the retriever can semantically locate the CBN MPC Communique highlighting the MPR remaining at a high 26.5% to combat 15.10% inflation [7], alongside the NGX corporate filing for the CP issuance.[29] The LLM can then successfully deduce that corporate debt yields were priced at a severe premium specifically to account for the Central Bank's aggressive liquidity tightening and the persistent inflationary environment.

8.2. Correlating Security, Hydrocarbons, and Fiscal Stability

A highly mature RAG system engineered with this pipeline can draw profound third-order analytical corollaries. By ingesting the ACLED data detailing localized kinetic events [47], alongside the NUPRC monthly production status reports [14], and the NBS Consumer Price Index [5], the model can highlight causal chains that human analysts might miss.

If the ACLED pipeline ingests data showing an uptick in militant demonstrations or vandalism in the Niger Delta, the RAG can automatically correlate this with a subsequent drop in the NUPRC's recorded swamp and shallow offshore production metrics. Because crude oil exports are Nigeria's primary foreign exchange earner, the model can logically connect this production deficit to FX liquidity shortages. This shortage directly explains the widening gap between the official exchange rate and the Binance P2P parallel market rate scraped by SeleniumBase.[51] Ultimately, the LLM can synthesize that this FX spread places upward pressure on the imported food index within the NBS CPI reports, mapping a direct line from a localized pipeline incident to national food inflation.[7]

8.3. Auditing the Refining Ecosystem

The ingestion of the 1st Half 2025 DCSO Refinery Requirement Forecast from the NUPRC provides the exact regulatory mandates for domestic crude allocation (770,500 bopd).[14] By cross-referencing this statutory requirement with the real-time AIS data tracking the volume of crude tankers entering the Lekki Deep Sea Port [43], and correlating both datasets against the EIA's global crack spreads [34], the RAG pipeline achieves a holistic, empirical view of the downstream ecosystem.

The LLM is no longer restricted to summarizing what the regulations dictate; it can answer questions about whether the physical movement of maritime assets indicates compliance with the DCSO, and whether current global refining margins financially incentivize domestic refining over international crude exportation.

9. Conclusion

Building a high-fidelity Retrieval-Augmented Generation pipeline capable of delivering deep economic, maritime, and energy intelligence requires moving entirely beyond standard, pre-packaged datasets. By engineering a custom, aggressive ingestion framework that targets primary institutional sources—the CBN, NBS, NNPC, DMO, and NUPRC—the pipeline secures

the foundational truth required for authoritative generation.
The implementation of stealth extraction libraries like curl_cffi and SeleniumBase neutralizes the severe friction introduced by modern Web Application Firewalls protecting these governmental portals from foreign datacenters in Ontario. Furthermore, incorporating parallel data streams, such as the HDX ACLED API for conflict data and continuous WebSocket telemetries for maritime movements, equips the final vector database with a dynamic, multi-dimensional perspective of the Nigerian landscape.
When these provided Python scripts are executed systematically via scheduled cron jobs or containerized orchestrators, they will continuously hydrate the knowledge base. This ensures the downstream generative models are not relying on stale training data, but are instead responding with the most current, empirically accurate, and highly contextualized intelligence available in the global market.

Works cited

1. How to Bypass Cloudflare When Web Scraping in 2026 - Scrapfly, accessed March 9, 2026, https://scrapfly.io/blog/posts/how-to-bypass-cloudflare-anti-scraping
2. How to Bypass Cloudflare in Python - ZenRows, accessed March 9, 2026, https://www.zenrows.com/blog/bypass-cloudflare-python
3. How to avoid a bot detection and scrape a website using python? - Stack Overflow, accessed March 9, 2026, https://stackoverflow.com/questions/68895582/how-to-avoid-a-bot-detection-and-scrape-a-website-using-python
4. How to Bypass Cloudflare in 2026: Top Methods & Scripts - Bright Data, accessed March 9, 2026, https://brightdata.com/blog/web-data/bypass-cloudflare
5. Consumer Price Index and Inflation CPI - microdata.nigerianstat.gov.ng., accessed March 9, 2026, https://microdata.nigerianstat.gov.ng/index.php/catalog/154
6. National Bureau of Statistics: Home, accessed March 9, 2026, https://www.nigerianstat.gov.ng/
7. Nigeria Inflation Rate - Trading Economics, accessed March 9, 2026, https://tradingeconomics.com/nigeria/inflation-cpi
8. Elibrary - National Bureau of Statistics, accessed March 9, 2026, https://nigerianstat.gov.ng/elibrary/search/cpi
9. Monetary Policy Decisions | Central Bank of Nigeria - CBN, accessed March 9, 2026, https://www.cbn.gov.ng/MonetaryPolicy/decisions.html
10. CBN Statistical Bulletin 2020 Edition | PDF | Gross Domestic Product | Banks - Scribd, accessed March 9, 2026, https://www.scribd.com/document/841955997/2020-Statistical-Bulletin-Explanatory-Notes-Final-Modified
11. Nigeria's Total Public Debt as at June 30, 2025 - DMO, accessed March 9, 2026, https://dmo.gov.ng/debt-profile/total-public-debt/5541-nigeria-s-total-public-debt-as-at-june-30-2025
12. Nigeria's Total Public Debt as at December 31, 2024, accessed March 9, 2026,

https://www.dmo.gov.ng/debt-profile/total-public-debt/5215-total-public-debt-as-at-december-31-2024
13. discussion paper - Open Knowledge Repository, accessed March 9, 2026, https://openknowledge.worldbank.org/bitstreams/3ba4c2b7-5b54-5420-9bad-874093135928/download
14. Gazetted Regulations - Nigerian Upstream Petroleum Regulatory Commission, accessed March 9, 2026, https://www.nuprc.gov.ng/gazetted-regulations/
15. Download - Nigerian Upstream Petroleum Regulatory Commission, accessed March 9, 2026, https://www.nuprc.gov.ng/wp-content/uploads/2025/06/NIGERIAN-UPSTREAM-PETROLEUM-COMMERCIAL-REGULATIONS.pdf
16. Investors - NNPC LTD, accessed March 9, 2026, https://www.nnpcgroup.com/investors
17. Filesystem - 2024 - NNPC LTD, accessed March 9, 2026, https://www.nnpcgroup.com/documents/Investor%20Documents/2024
18. PRESS RELEASE: NNPC Limited Declares ₦5.4 Trillion Profit After Tax, accessed March 9, 2026, https://nnpcgroup.com/insights/press-release-nnpc-limited-declares-5-4-trillion-profit-after-tax
19. Annual Financial Report, accessed March 9, 2026, https://cms1977.nnpcgroup.com/uploads/2024_Annual_Report_669bbd1ef7.pdf
20. NNPC Releases 2024 Audited Financial Statement, Declares N5.4 Trillion Profit, N45.1tn Revenue - THISDAYLIVE, accessed March 9, 2026, https://www.thisdaylive.com/2025/11/25/nnpc-releases-2024-audited-financial-statement-declares-n5-4-trillion-profit-n45-1tn-revenue/
21. Nigeria Oil and Gas Audit Report 2022 | PDF - Scribd, accessed March 9, 2026, https://www.scribd.com/document/850212298/Final-Report-NEITI-OGA-2022-Final-26-Sept-2024
22. Oil And Gas Industry Audit 2022 - NGF Digital Repository, accessed March 9, 2026, https://ngfrepository.org.ng:8443/jspui/handle/123456789/6663
23. FINAL REPORT - EITI, accessed March 9, 2026, https://eiti.org/sites/default/files/2024-09/Final%20Report_NEITI_OGA_2023_Final_26_Sept_2024.pdf
24. NNPC-Audited-Report.pdf - Fiscal Transparency, accessed March 9, 2026, https://www.fiscaltransparency.org/wp-content/uploads/2025/03/NNPC-Audited-Report.pdf
25. NMDPRA - Leading Oil & Gas Regulatory Authority in Nigeria | Ensuring Development through Sustainable Energy, accessed March 9, 2026, https://www.nmdpra.gov.ng/
26. NMDPRA Proposed Draft Regulations - Petroleum Industry Act, accessed March 9, 2026, https://pia.gov.ng/typography-styles/
27. GUIDELINES FOR THE IMPORTATION OF PETROLEUM PRODUCTS INTO NIGERIA - NMDPRA-IMPEX, accessed March 9, 2026, https://impex.nmdpra.gov.ng/Manuals/GUIDELINES.pdf
28. Dangote Refinery IPO in sight as oil & gas sector leads NGX rally - CNBC Africa,

accessed March 9, 2026, https://www.cnbcafrica.com/media/7771856694990/dangote-refinery-ipo-in-sight-as-oil-gas-sector-leads-ngx-rally-
29. NGX records first Commercial Paper listing with Dangote Cement's N119.9bn issuance, accessed March 9, 2026, https://www.vanguardngr.com/2026/02/ngx-records-first-commercial-paper-listing-with-dangote-cements-n119-9bn-issuance/
30. X-DataPortal - Nigerian Exchange Limited - NGX, accessed March 9, 2026, https://ngxgroup.com/exchange/data/x-dataportal/
31. Monthly Oil Market Report (MOMR) - OPEC.org, accessed March 9, 2026, https://www.opec.org/monthly-oil-market-report.html
32. Monthly Oil Market Report - OPEC Digital Publications, accessed March 9, 2026, https://publications.opec.org/momr
33. Oil Market Report - February 2026 – Analysis - IEA, accessed March 9, 2026, https://www.iea.org/reports/oil-market-report-february-2026
34. What drives petroleum product prices: Prices and Crack Spreads - U.S. Energy Information Administration (EIA), accessed March 9, 2026, https://www.eia.gov/finance/markets/products/prices.php
35. Spot Prices for Crude Oil and Petroleum Products - EIA, accessed March 9, 2026, https://www.eia.gov/dnav/pet/pet_pri_spt_s1_d.htm
36. Oil Price API, accessed March 9, 2026, https://www.oilpriceapi.com/
37. Real-Time & Historical Prices for Crude Oil, Gold, Silver, Coffee & More. - Commodities API, accessed March 9, 2026, https://commodities-api.com/?k=207906618
38. Energy Trading API | Real-Time Oil & Gas Prices for Traders, accessed March 9, 2026, https://www.oilpriceapi.com/solutions/energy-trading
39. aisstream.io, accessed March 9, 2026, https://aisstream.io/
40. AIS Stream API Reference - aisstream.io, accessed March 9, 2026, https://aisstream.io/documentation
41. VesselFinder: Ship & Container Tracking, accessed March 9, 2026, https://www.vesselfinder.com/
42. Free AIS vessel tracking | AIS data exchange | JSON/XML ship positions, accessed March 9, 2026, https://www.aishub.net/
43. Vessel Schedule - Lekki Freeport Terminal, accessed March 9, 2026, https://www.lft-ng.com/tools/vesselSchedule
44. Port of Lekki (Nigeria) - Arrivals, Departures, Expected vessels - VesselFinder, accessed March 9, 2026, https://www.vesselfinder.com/ports/NGLKK002
45. LEKKI FREEPORT TERMINAL LAGOS - Container Terminal Information & Tracking, accessed March 9, 2026, https://terminal49.com/terminals/lftl-lekki-freeport-terminal-lagos
46. Lekki/NGLKK Port - Container Tracking and Performance - Terminal49, accessed March 9, 2026, https://terminal49.com/ports/nglkk-lekki
47. Nigeria - Conflict Events | Humanitarian Dataset | HDX, accessed March 9, 2026, https://data.humdata.org/dataset/nigeria-acled-conflict-data
48. Armed Conflict Location & Event Data Project (ACLED) | HDX, accessed March 9,

2026, https://data.humdata.org/organization/acled
49. Getting started - ACLED, accessed March 9, 2026, https://acleddata.com/api-documentation/getting-started
50. GitHub - OCHA-DAP/hdx-python-api: Python API for interacting with the HDX Data Portal, accessed March 9, 2026, https://github.com/OCHA-DAP/hdx-python-api
51. This is a simple binance p2p scraper project that send notifications by email - GitHub, accessed March 9, 2026, https://github.com/dhsosa98/ts-binance-p2p-scraper